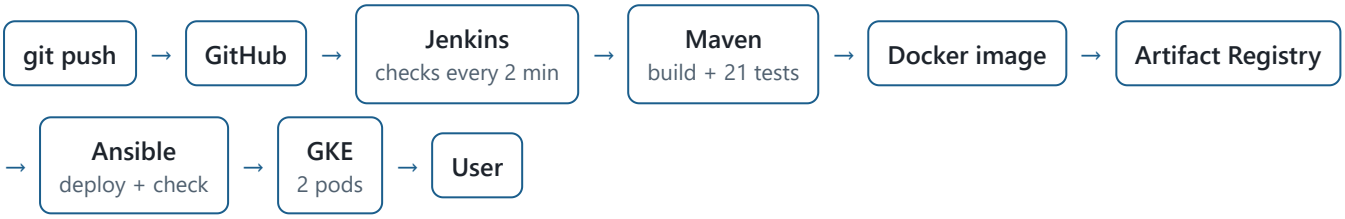
CIA 2 Study Guide

Spring Boot + Maven + Docker + Jenkins + Ansible + Kubernetes on Google Cloud (GKE)

Pranav Surya R S · personal study notes

1. The big picture

Goal: when code is pushed to Git, the app is built, tested, put in a container and deployed to Kubernetes in the cloud, with no manual steps.



1. You push to GitHub (`pranavsurya77/devops-lab-cia2` , branch `main`).
2. Jenkins (on your laptop) checks GitHub every 2 minutes, sees the new commit and starts a build.
3. **Build and Test:** Maven compiles, runs 21 tests, makes the jar. A failed test stops everything.
4. **Docker Build and Push:** the jar goes into an image tagged with the build number, uploaded to Artifact Registry.
5. **Deploy with Ansible:** puts the new image in `deployment.yaml` , runs `kubectl apply` , waits for pods, checks `/actuator/health` .
6. Kubernetes swaps old pods for new ones one at a time. Same public IP, new version.

> **One-line answer:** Every push to GitHub triggers a Jenkins pipeline that builds and tests the Spring Boot app with Maven, packs it into a Docker image in Google Artifact Registry, and deploys it to a GKE Autopilot cluster with an Ansible playbook that also checks the app is healthy.

Build #3 (successful): Checkout 2 s · Build and Test 16 s · Docker 4 min 15 s · Deploy 3 min · total 7 min 39 s.

2. Key words

Term	Meaning
CI / CD	Auto build and test every push / auto release what passes.
Image / Container	Image = packed app + Java, built from a Dockerfile. Container = a running copy of an image.
Artifact / Registry	Artifact = any build output (jar, image). Registry = where images are stored for others to download.
Kubernetes	Runs containers, restarts them if they crash, updates them without downtime.
Cluster / Node / Pod	Cluster = group of machines. Node = one machine. Pod = smallest thing K8s runs, holds our container.

Deployment / Service	Deployment keeps N pods running. Service gives them one fixed address and spreads traffic.
Ansible / Playbook / Inventory	Automation tool / the YAML file of tasks / the list of machines to run on.
Jenkinsfile	The pipeline written as code, kept in the repo.
Service account	A Google Cloud "robot user" for programs (ours: `cia2-github`).

3. The app and Maven

Library Book Manager: Spring Boot 3.3.5 REST API, Java 21. No UI. Books kept in memory (3 sample books). Main endpoints:

- `/` or `/api/info`: version and host. On Kubernetes the host is the **pod name**.
- `/api/books`: list, add, get, borrow (409 if already borrowed), return, delete.
- `/actuator/health`: returns `{"status":"UP"}`. Ansible uses it to confirm the deploy.
- `/swagger-ui.html`: try every endpoint in the browser.

Tests (21): 10 for the borrow/return rules, 8 for HTTP status codes, 3 that start the whole app.

application.properties: `server.port=8080` (the app's port), `app.version=1.0.0` (shown at `/`), and the two `management...` lines expose only the health and info endpoints.

Jenkins runs `mvn -B -ntp clean package`: `-B` batch mode (no prompts), `-ntp` hides download progress, `clean` deletes old output, `package` = compile → test → build `target/library-manager.jar`. This jar includes all libraries and a web server, so `java -jar` is enough to run it.

4. Dockerfile and .dockerignore

#	Line	Meaning
1	`FROM eclipse-temurin:21-jre-alpine`	Start from an image that has Java 21. Alpine is tiny Linux: 342 MB instead of 512 MB, fits the free registry storage.
3	`COPY target/library-manager.jar app.jar`	Copy the jar Maven built into the image.
5	`EXPOSE 8080`	Notes that the app listens on 8080.
7	`ENTRYPOINT ["java", "-jar", "app.jar"]`	Command run when the container starts.

.dockerignore has `*` (ignore everything) and `!target/library-manager.jar` (except the jar). So only the jar is sent to Docker: faster, and no source code or secrets in the image.

Why so simple: Jenkins already builds the jar, so Docker only needs to copy and run it.

5. Artifact Registry

An **artifact** is a file a build produces (jar, Docker image). **Artifact Registry** is Google Cloud's private storage for them. We need it because the image is built on your laptop but the pods run on Google's machines, which must download it from somewhere. It is in the same project and region as the cluster, so GKE pulls without extra setup.

```
asia-south1-docker.pkg.dev / cia2-513606 / library-manager / library-manager : 3
   registry (region)          project ID     repository         image        tag = build number
```

Each build adds a new tag (1, 2, 3...), so every version is kept and traceable. The console shows 95.9 MB because images are stored compressed.

6. Kubernetes and GKE

GKE Autopilot: Google runs Kubernetes and the machines (nodes) for us; we only describe pods. Cluster `cia2-gke`, region `asia-south1` (Mumbai), project `cia2-513606`. The GKE free tier covers the cluster fee; pods and the load balancer use the credit.

> **How many pods? 2**, because `replicas: 2`. If one crashes, Kubernetes starts a new one. During an update there are briefly **3**: it starts 1 new pod, waits until it's ready, removes 1 old one, and repeats (rolling update, no downtime).

Request path: browser → public IP `35.200.213.28` port 80 → Service → one of the 2 pods on port 8080 → Spring Boot. The Service finds pods by their **label** `app: library-manager`, so it works even though pod names change on every deploy.

deployment.yaml line by line

#	Line	Meaning
1	`apiVersion: apps/v1`	API version for Deployments.
2	`kind: Deployment`	Object type: keeps pods running and handles updates.
3-4	`metadata:` `  name: library-manager`	Its name, used in `kubectl` commands.
5	`spec:`	What we want.
6	`  replicas: 2`	Always keep 2 pods.

7-9	`  selector:`
	`  matchLabels:`
	`  app: library-manager`
		This Deployment owns pods with label `app=library-manager`. Must match line 13.

10	`  template:`	Blueprint for each pod.

11-13	`  metadata:`
	`  labels:`
	`  app: library-manager`
		The label every pod gets. The Service uses it too.

14-16	`  spec:`
	`  containers:`
	`  - name: library-manager`
		One container per pod, with this name.

17	`  image: library-manager:local`	**Placeholder.** Ansible replaces it with the real image and tag on each deploy. A new image is what triggers the rolling update.
18-19	`  ports:` `  - containerPort: 8080`	The container listens on 8080.

service.yaml line by line

#	Line	Meaning
1-2	`apiVersion: v1` `kind: Service`	A Service: one fixed address in front of the pods.
4	`  name: library-manager`	Its name. Ansible reads the IP from it.
6	`  type: LoadBalancer`	Get a public IP from Google Cloud. (`ClusterIP` would be inside-only.)
7-8	`  selector:` `    app: library-manager`	Send traffic to pods with this label (our 2 pods).
10	`    - port: 80`	Port users connect to (normal HTTP, so no port in the URL).
11	`      targetPort: 8080`	Port on the pod where Spring Boot listens.

7. Ansible

What: an automation tool. You list tasks in a YAML **playbook** and it runs them in order, the same way every time. No agent needed on target machines.

Why: deploying is 5 steps (set image, apply, wait, get IP, health check). A playbook makes them repeatable and readable instead of loose commands.

How we set it up: Ansible doesn't run on Windows, and Jenkins runs on Windows. So each deploy starts a Linux container (`google/cloud-sdk:slim`), installs Ansible in it and runs the playbook there. Nothing is installed on your laptop.

inventory.ini

#	Line	Meaning
1	`[local]`	A group called `local` .
2	`localhost ansible_connection=local`	Run on this machine (the container) directly, no SSH. We only need it to run `kubectl` .

deploy.yml line by line

#	Line	Meaning
1-2	`- name: Deploy Library Manager to Kubernetes` `  hosts: local`	A play, run on the `local` group.

3-5	`  vars:`
	`    image: library-manager:local`
	`    k8s_dir: "{{ playbook_dir }}/../k8s"`
		Default image (Jenkins overrides it with `-e image=...`) and the path to the k8s folder. `{{ }}` inserts a variable.

8-12	`ansible.builtin.replace:`
	`  path: ".../deployment.yaml"`
	`  regexp: "image: .*"`
	`  replace: "image: {{ image }}"`
		Find the `image:` line and replace it with the new image.

14-15	`command: kubectl apply -f {{ k8s_dir }}`	Apply both YAML files: create if missing, update if changed.
17-18	`command: kubectl rollout status deployment/library-manager --timeout=600s`	Wait until the new pods are ready (max 10 min).

20-25	`command: kubectl get service library-manager -o jsonpath={...ip}`
	`register: ip`
	`until: ip.stdout != ""`
	`retries: 30`
	`delay: 10`
		Read the public IP and save it in `ip`. Retry every 10 s, up to 30 times, until the IP exists.

27-33	`ansible.builtin.uri:`
	`  url: "http://{{ ip.stdout }}/actuator/health"`
	`until: health.status == 200`
		Call the health URL (like curl) until it returns 200. This proves the app is live.

35-37	`ansible.builtin.debug:` `  msg: "App is running at http://..."`	Print the app URL in the log.

PLAY RECAP `ok=7 changed=4 failed=0`**:** 7 tasks OK (6 ours + "Gathering Facts"), 4 report "changed" (every `command` task always does), 0 failed.

deploy.sh line by line (runs inside the container)

#	Line	Meaning
3	`set -e`	Stop at the first error, so Jenkins marks the stage failed.
6	`cp -r /work /tmp/app && cd /tmp/app`	Work on a copy. Ansible could not edit the Windows-mounted folder directly (build #1 failed on this).
8	`apt-get install ... ansible-core kubectl google-cloud-cli-gke-gcloud-auth-plugin`	Install Ansible, kubectl, and the plugin kubectl needs to log in to GKE.
10	`gcloud auth activate-service-account --key-file=/key.json`	Log in as the `cia2-github` service account with its key.
11	`gcloud container clusters get-credentials cia2-gke --region asia-south1 ...`	Point `kubectl` at our cluster.
13	`ansible-playbook -i ansible/inventory.ini ansible/deploy.yml -e image="$1"`	Run the playbook. `$1` is the image name Jenkins passed in.

8. Jenkins and the Jenkinsfile

Jenkins 2.568.2 runs on your laptop as a Windows service at `http://localhost:8080`. Setup:

1. Added the service account JSON key as a **Secret file** credential, ID `gcp-key`.
2. Created a **Pipeline** job `cia2-pipeline`, "Pipeline script from SCM", pointing at the GitHub repo, branch `main`, file `Jenkinsfile`.
3. Clicked Build Now once to register the 2-minute polling. After that, pushes trigger builds.

#	Line	Meaning
1-2	`pipeline {` `  agent any`	A declarative pipeline, run on any available agent (our laptop).

4-7	`triggers {`
	`  pollSCM('H/2 * * * *')`
	`}`
		Check GitHub every 2 minutes; build if there's a new commit. Polling, because a laptop has no public URL for a GitHub webhook.

10	`PATH = "C:\\tools\\apache-maven...;...DockerDesktop\\resources\\bin;${env.PATH}"`	Add Maven and Docker to PATH. The Jenkins service doesn't have your user PATH, so without this `mvn` and `docker` aren't found.
11	`IMAGE = "asia-south1-docker.pkg.dev/.../library-manager:${BUILD_NUMBER}"`	Full image name. `BUILD_NUMBER` (1, 2, 3...) makes each tag unique.
12	`GCP_KEY = credentials('gcp-key')`	Gives the path to the secret key file. Shown as `****` in logs.

16-20	`stage('Build and Test') {`
	`  bat 'mvn -B -ntp clean package'`
	`}`
		Compile, test, build the jar. `bat` = run a Windows command.

22-28	`stage('Docker Build and Push') {`
	`  bat 'docker build -t %IMAGE% .'`
	`  bat 'type "%GCP_KEY%" | docker login -u _json_key --password-stdin https://asia-south1-docker.pkg.dev'`
	`  bat 'docker push %IMAGE%'`
	`}`
		Build the image. Log in to Artifact Registry by feeding the key file as the password (`_json_key` is Google's username for key login). Push the image. `%IMAGE%` reads a variable in Windows.

30-34	`stage('Deploy with Ansible') {`
	`  bat 'docker run --rm -v "%WORKSPACE%:/work" -v "%GCP_KEY%:/key.json" -w /work google/cloud-sdk:slim bash ansible/deploy.sh %IMAGE%'`
	`}`
		Start a Linux container and run `deploy.sh` . `--rm` deletes it after. `-v` mounts the repo at `/work` and the key at `/key.json` . `-w` sets the start folder.

Jenkins also adds a **Checkout SCM** stage first, which clones the repo. We used GitHub Actions at first, then moved to Jenkins to keep the pipeline simple and local.

9. Google Cloud setup, problems, security

Setup in the console: project `cia2-513606` → budget alert → enabled Kubernetes Engine and Artifact Registry APIs → created Docker repo `library-manager` (asia-south1) → created Autopilot cluster `cia2-gke` → service account `cia2-github` with a JSON key and only 2 roles: **Artifact Registry Writer** (push images) and **Kubernetes Engine Developer** (deploy).

Problem	Fix
Port 8080 already used by Jenkins	Ran the app locally on 8081
Image push "permission denied"	Gave the service account its 2 roles
Jenkins couldn't find mvn / docker	Added them to PATH in the Jenkinsfile
Ansible doesn't run on Windows	Ran it in a Linux container
Ansible couldn't edit deployment.yaml	Copied files inside the container first

Security: key never in Git, stored as a Jenkins secret and hidden in logs; service account has only 2 roles; image holds only the jar.

Limits: data in memory (lost on restart); plain HTTP, no login; no health probes in deployment.yaml (kept simple); polling instead of webhook.
Stop costs: scale the workload to 0 and delete the Service, or delete the cluster. A new Jenkins build brings it all back.

10. Viva questions

How is the pipeline triggered? Jenkins polls GitHub every 2 minutes (`pollSCM('H/2 * * * *')`) and builds on a new commit.

Why not a webhook? GitHub would need to reach Jenkins over the internet; a laptop has no public URL.

What if a test fails? Maven fails, the pipeline stops, nothing is built or deployed.

How many pods and why? 2 (`replicas: 2`), so one can fail and updates have no downtime. Briefly 3 during an update.

Deployment vs Service? Deployment runs and updates the pods. Service gives them one stable address and load-balances.

How does the Service find pods? By the label `app: library-manager` .

Port 80 vs 8080? Users use 80 on the load balancer; it forwards to 8080 on the pod, where Spring Boot listens.

Why LoadBalancer? To get a public IP. ClusterIP is only reachable inside the cluster.

What is Artifact Registry? Google Cloud's private storage for build outputs; we keep Docker images there so GKE can pull them.

How is a new version deployed? New image tag (build number) → Ansible writes it into deployment.yaml → `kubectl apply` → rolling update.

What is a rolling update? Replace pods one at a time so the app never goes down.

Why Ansible, and why in a container? To make deploy steps repeatable. In a container because Ansible doesn't run on Windows.

What do until / retries / delay do? Repeat a task until it works: up to 30 tries, 10 s apart.

How does Jenkins get access to Google Cloud? The `cia2-github` service account key, stored as Jenkins credential `gcp-key` , with only 2 roles.

Why Alpine and a copy-only Dockerfile? Smaller image (342 MB vs 512 MB); Jenkins already built the jar.

What is GKE Autopilot? GKE where Google manages the machines; we only define pods.

How do you verify the deploy? `kubectl rollout status` , then Ansible checks `/actuator/health` returns 200.

What if a pod crashes? The Deployment starts a replacement; the Service stops sending it traffic.

What would you improve? A database, health probes, HTTPS, a webhook, and no long-lived key (Workload Identity).